

# Embedded projects 2

## Weekly exercises 3&4

Vihtori Virta  
Sonja Viinikainen

# Exercise 1

As with most sensors, there can be error values at times, so it is not wise to trust a single value reading. Because of this, average out the LiDAR data by taking at least **x** samples and calculating their average:

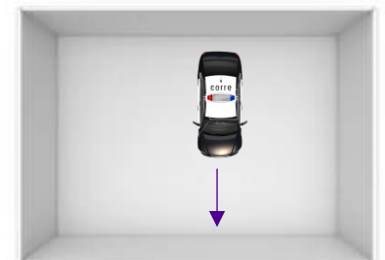
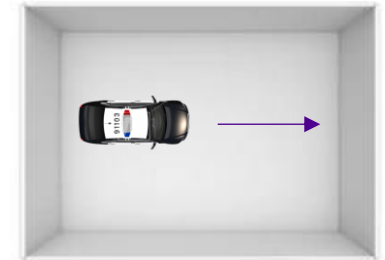
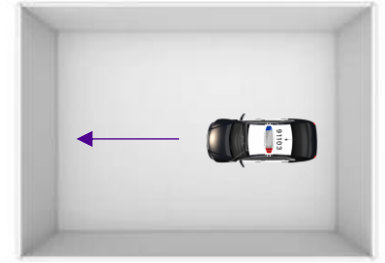
- Create a new function for averaging the values. This function should:
  - Accept the number of samples (**x**) as an argument
  - Calculate and return the averaged distance
  - Define **x** in your code and test what amount of samples is giving stable lidar results (10-100 maybe?)
- Call this function from the main function, passing the desired number of samples to average.
- Print the averaged LiDAR distance (in cm) onto your LCD at a frequency of 1 Hz.

Note, this exercise does not require any of the other functionalities like ESP or such

## Exercise 2

- Create a program which using the lidar, measures a walled space's area ( $\text{m}^2$ ) and volume ( $\text{m}^3$ )
  - You can assume the space is rectangular and ceiling height is 100 cm
  - Use averaged lidar distance values (from exercise 1)
  - Rotate your car in a way, that the car does not move too much, for example turn right wheel backwards, and left wheel forwards when turning
- Use serial monitor or a button to start the measurements
- Results for each **four** directions, area and volume are printed onto the LCD

Car in a box

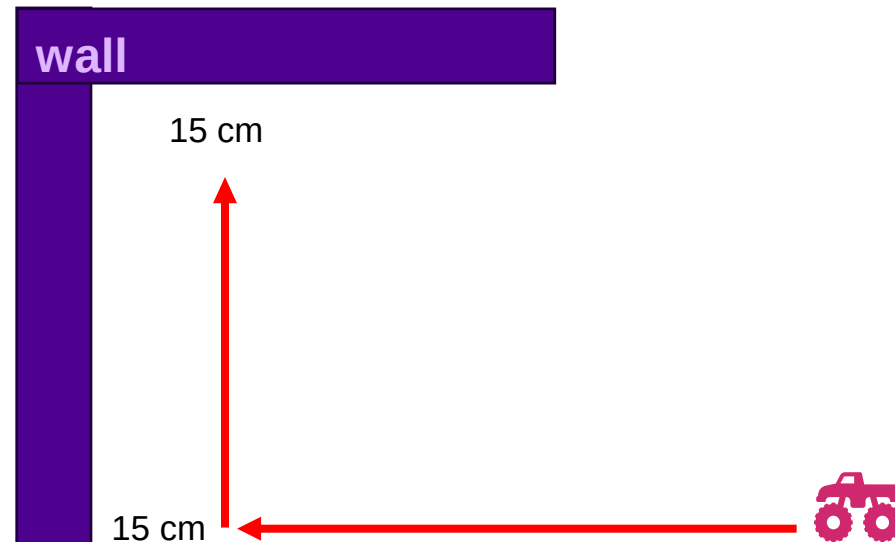


# Exercise 3

- Step 1: Create a program that drives your car to a set distance from object/wall
  - Set fixed distance to 30 cm
  - You give a "follow" command with a button or a command to the serial bus
  - Your car will drive forwards/backwards so that distance between car and object is 30 cm
  - Your car keeps the same distance between a moving object (30cm) even if the object moves forwards or backwards

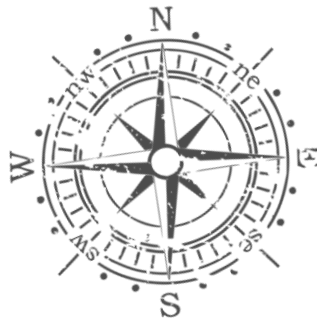
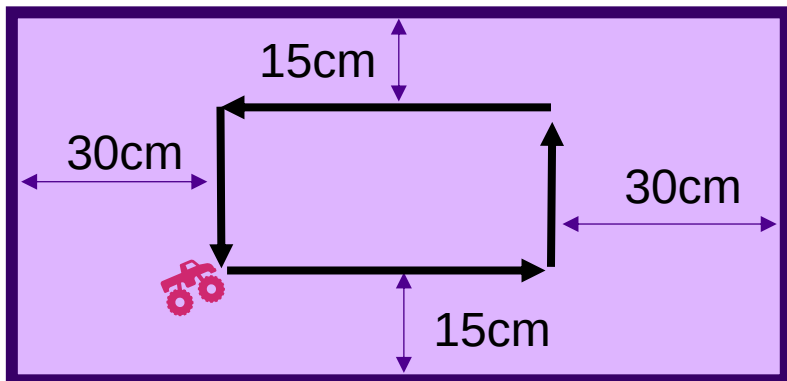
## Exercise 4

- Step 1: Create a program that drives your car forward, until there is a distance of 15 cm's from the wall. After reaching that value, the car rotates 90 degrees right and then drives again until at the same distance from the second wall
- Step 2: If the starting distance is less that 15 cm, you must reverse to reach the distance of 15 cm's. If there is no wall closer than 1 meter, do not move.



## Exercise 5

- Create a program that uses lidar and compass to drive your car according to the path below (pink car symbol shows where route starts and ends)
  - Measure the total distance driven (with encoder) and print it onto the LCD
  - You can assume that left is west, up is north, ...
- Make sure your robot autocorrects the direction according to the compass value, if the direction starts drifting



# Exercise 6 - Calibration of Encoders Using Lidar Assistance

**Objective:** The goal is to calibrate how much the car moves between encoder pulses and understand how the distance traveled by the car may vary with the same pulse count on different surfaces, such as good traction versus a more slippery surface.

## Procedure:

1. Drive the car straight forward based on LiDAR measurements for a distance of 20 cm. This distance serves as a reference for calibrating the driving distance based on encoder pulses.
2. Collect the number of pulses generated by the encoder during the reference drive of 20 cm.

## Calibration:

Based on the pulses and the traveled distance, calculate the length driven for one pulse. Print the obtained result, for example, to the serial monitor. Test the pulse length on different driving surfaces (e.g., plywood vs. the floor). Save the result to EEPROM so that the length can be read from memory every time the system restarts. See next slide for help, and note that you do not need for example ESP-code for this task.

## Exercise 6 – Hints

- You can first start with just testing how writing and reading works with [EEPROM](#)
  - Typically, it is better to use function [update](#) instead of put or write
- Below is a little bit of pseudo code if you feel stuck

```
Setup:
  Read EEPROM Value
  Print currently stored value onto LCD

Main Loop:
  Call Calibration (with serial command or button...)

Calibration:
  **Here, add the measuring functionality**

  Print Result to Serial Monitor
  Save Result to EEPROM
```